IT CAREER HUB

Auto QA

Введение в Requests



Преподаватель



Роман Сурков

Full Stack QA

Опыт работы с **2020**

Тестировал **мобильные**, **веб** приложения, а также **AR**

На данный момент работаю во французской компании **ABTasty**

Достижения:

- Выстраивал QA процессы с нуля
- Внедрил **API** и **E2E** авто-тесты
- Руководил небольшой командой и нанимал Junior QA
- Обучил 100+ студентов
- Веду блог на Linkedin

surkov.ron@gmail.com

План занятия

- Работа со словарями
- Основные моменты работы со словарями
- Клиент-серверная архитектура
- Запросы в HTTP
- Получение списка компаний
- API Reference
- Создание компании
- Создание класса CompanyApi и использование его в тестах
- Зачем нужен CompanyApi?

ICH IT CAREER HUB

ОСНОВНОЙ БЛОК

IT CAREER HUB



Работа со словарями

Работа со словарями



В Python словарь (dict) хранит пары ключ-значение. Данные можно получать, изменять, добавлять и удалять.

```
space_mission = {
    "Название": "Artemis I",
    "Год запуска": 2022,
    "Страна": "США",
    "Экипаж": None,  # Беспилотная миссия
    "Продолжительность (дней)": 25,
    "Оборудование": ["Орион", "Сервисный модуль", "Манекены с датчиками"],
```

Создание словаря

```
    "Результаты": {
        "Дальность полёта (км)": 432000,
        "Тестирование систем": "Успешно",
        "Возвращение на Землю": True,
        "Данные о радиации": {
            "Общая доза (мЗв)": 12.5,
            "Максимум на орбите Луны (мЗв)": 8.2
        }
    }
}
```

Создание словаря

```
print(space_mission["Название"])  # Artemis I

print(space_mission["Год запуска"])  # 2022
```

Доступ к элементам

```
print(space_mission["Оборудование"][1])  # Сервисный модуль
```

Список в словаре

```
print(space_mission["Результаты"]["Дальность полёта (км)"])  # 432000

print(space_mission["Результаты"]["Данные о радиации"]["Максимум на орбите Луны (мЗв)"])  # 8.2
```

Вложенные словари

```
space_mission["Продолжительность (дней)"] = 30  # Меняем продолжительность миссии

space_mission["Результаты"]["Тестирование систем"] = "Частично успешно"  # Меняем результат теста
```

Изменение значений

```
space_mission["Цель"] = "Испытание технологий для полётов на Луну"

space_mission["Результаты"]["Температура корпуса"] = "Средняя: 50°C, Максимальная: 80°C"
```

Добавление новых данных

```
del space_mission["Экипаж"]  # Удаляем экипаж (т.к. беспилотная миссия)

удаленное = space_mission["Результаты"].pop("Возвращение на Землю", "Нет данных")  # Удаляем и получаем значение
```

Удаление элементов

```
for key, value in space_mission.items():

    print(f"{key}: {value}")
```

Вывод всех ключей и значений

```
for key, value in space_mission["Результаты"].items():

    print(f"{key}: {value}")
```

Перебор вложенного словаря

```python
if "Цель" in space_mission:
    print("Данные о цели миссии есть")
```

Проверка наличия ключа

```
копия = space_mission.copy()  # Поверхностная копия
```

Копирование словаря

Основные моменты работы со словарями

- **Ключи** – уникальные, могут быть строками, числами.
- **Значения** – любые данные (числа, строки, списки, другие словари).
- **Доступ** – `dict[key]`, `.get()`.
- **Изменение** – просто присваиваем новое значение.
- **Перебор** – `.items()`, `.keys()`, `.values()`.
- **Удаление** – `del`, `.pop()`.

IT CAREER HUB

ВОПРОСЫ

IT CAREER HUB



Основные моменты работы со словарями

Работа с JSON в Python



Сходства и различия JSON и Python

Синтаксис Python и JSON очень похож, но есть несколько отличий:

- В JSON логические значения пишутся с маленькой буквы: `true` и `false`, а в Python — с большой: `True` и `False`.
- В JSON отсутствие значения обозначается `null`, а в Python — `None`.

Преобразование JSON в словарь Python



Если мы хотим обрабатывать JSON в Python, можно работать с ним как со словарём. Для этого используем встроенный модуль `json` и его метод `loads()`, который превращает строку с JSON в словарь. Этот навык полезен при написании тестов для API, так как данные в запросах и ответах часто передаются в формате JSON.

Работа с JSON-объектом (словарём)



Допустим, у нас есть JSON-текст, который по структуре напоминает словарь.

```
import json  # Подключаем модуль json


company_json = """
{
    "id": 111,
    "isActive": true,
    "createDateTime": "2024-04-05T17:30:00.713Z",
    "lastChangedDateTime": "2024-04-05T17:30:00.713Z",
```

Работа с JSON-объектом (словарём)

```
 "name": "Барбершоп 'Цирюльникъ'",
    "description": "Крутые стрижки для крутых шишек"
}
"""

def test_parse_json():
    company = json.loads(company_json)  # Преобразуем JSON в словарь
    assert company["id"] == 111  # Проверяем значение ключа "id"
```

Работа с JSON-объектом (словарём)

Работа с массивом JSON-объектов (список словарей)



Рассмотрим JSON-данные, содержащие список компаний.

Что делает json.loads()?



Этот метод принимает на вход JSON-строку и возвращает обычный словарь Python, с которым можно работать.

```
company_list_json = """
[
    {
        "id": 111,
        "isActive": true,
        "createDateTime": "2024-04-05T17:30:00.713Z",
        "lastChangedDateTime": "2024-04-05T17:30:00.713Z",
        "name": "Барбершоп 'Цирюльникъ'",
        "description": "Крутые стрижки для крутых шишек"
    },
```

Работа с массивом JSON-объектов

```
{
    "id": 112,
    "isActive": true,
    "createDateTime": "2024-04-05T17:30:00.713Z",
    "lastChangedDateTime": "2024-04-05T17:30:00.713Z",
    "name": "Кондитерская Профи-троли",
    "description": "Сладко и точка"
},
```

Работа с массивом JSON-объектов

```
    {
        "id": 113,
        "isActive": true,
        "createDateTime": "2024-04-05T17:30:00.713Z",
        "lastChangedDateTime": "2024-04-05T17:30:00.713Z",
        "name": "Муж на час",
        "description": "Помощь в делах"
    }
]
```

Работа с массивом JSON-объектов

```
"""


def test_parse_array_json():
    company_list = json.loads(company_list_json)  # Преобразуем JSON в список словарей
    assert len(company_list) == 3  # Проверяем длину списка
```

Работа с массивом JSON-объектов

Важно!



После преобразования в Python `company_list` становится **списком словарей**.

Доступ к данным внутри списка



Так как `company_list` — это список, доступ к конкретному элементу осуществляется через индекс.

```python
def test_parse_array_json():
    company_list = json.loads(company_list_json)

    first_company = company_list[0]  # Первая компания (индекс 0)
    assert first_company["name"] == "Барбершоп 'Цирюльникъ'"  # Проверяем название компании

    # Проверяем ID второй компании
    assert company_list[1]["id"] == 112
```

Работа с массивом JSON-объектов

Итог

- JSON-объект (словарь) в Python превращается в обычный `dict`.
- JSON-массив (список объектов) превращается в `list` со словарями внутри.
- Для работы с JSON используем `json.loads()`, который превращает строку JSON в объект Python.
- Теперь мы можем извлекать и проверять данные при тестировании API.

Что произойдёт, если json.loads() получит некорректный JSON?



Если `json.loads()` получит некорректный JSON, он выбросит исключение `json.JSONDecodeError`.

```python
import json

invalid_json = "{ 'id': 111, 'name': 'Test Company' }"  # Неправильные одинарные кавычки

try:
    data = json.loads(invalid_json)
except json.JSONDecodeError as e:
    print(f"Ошибка при разборе JSON: {e}")
```

Пример ошибки

Частые ошибки в JSON

- Использование одинарных кавычек `'` вместо двойных `"` для строк.
- Отсутствие запятой между элементами.
- Лишняя запятая перед закрывающей `}` или `]`.
- Неправильное использование типов (`true` вместо `True`, `null` вместо `None`).

Как избежать ошибок?



Перед загрузкой JSON стоит проверять его валидность с помощью онлайн-проверок (например, jsonlint.com) или писать тесты.

IT CAREER HUB

ВОПРОСЫ

IT CAREER HUB



Клиент-серверная архитектура

Что такое клиент и сервер?



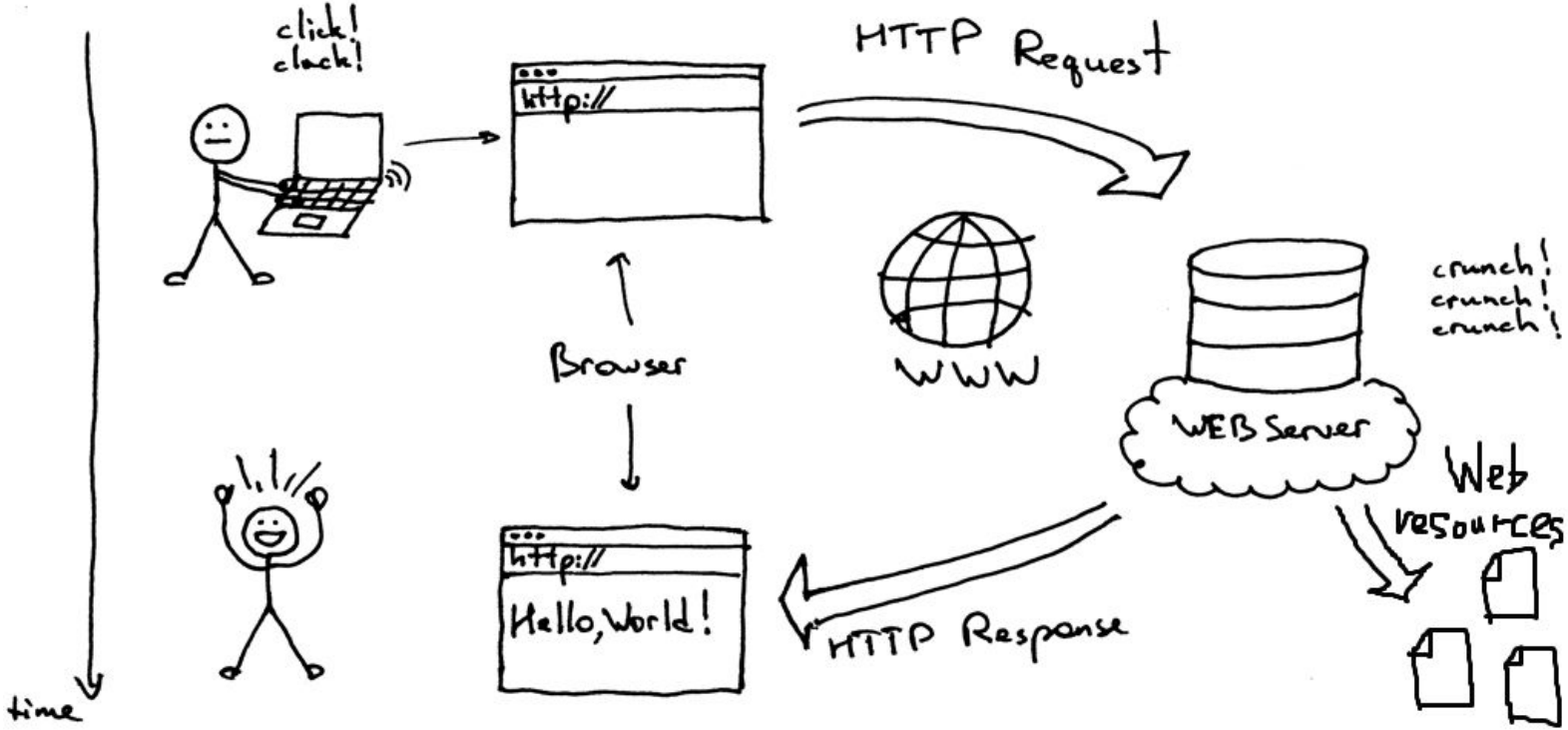
Клиент

Это устройство или программа, отправляющая запросы на сервер и получающая ответы (например, браузеры, мобильные приложения, умные устройства).

Сервер

Это компьютер, который принимает запросы, обрабатывает их и отправляет ответы.

Клиент и сервер



Важно



Клиентов может быть много, а сервер — один.


How API works
Web App

HTTP и HTTPS



HTTP

(HyperText Transfer Protocol) — протокол передачи гипертекста.

HTTPS

(HyperText Transfer Protocol Secure) — защищённая версия HTTP, использующая шифрование данных.

Разница между HTTP и HTTPS



HTTPS защищает данные от перехвата, что особенно важно для передачи паролей, банковских данных и другой конфиденциальной информации.

HTTPS

Поставщики сетей будут видеть ваши HTTPS следующим образом:

Domain: secure-example.com

IP Address: 192.0.2.123 Port: 443 Protocol: TLS 1.3

Timestamp: 22-Nov-2024 12:00:00 Packet Size: 1.2 KB

Encrypted Data: fkdj83kfa#$2jslkaf39fkjas...

НЕ увидят тело запроса и ответа, а также Headers + Endpoint

IT CAREER HUB

ВОПРОСЫ

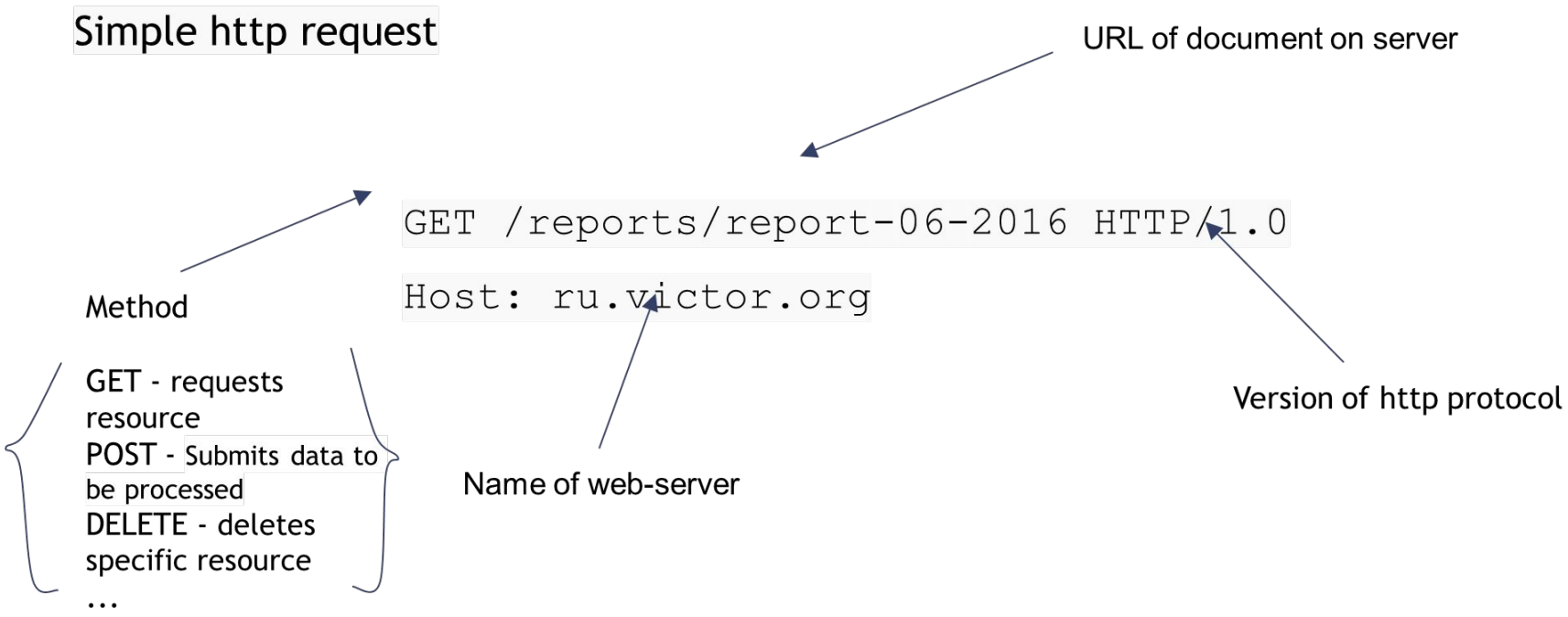
IT CAREER HUB

Запросы в HTTP

Из чего состоит HTTP-запрос?

- **URL** (*Uniform Resource Locator*) — адрес ресурса в интернете.
- **Метод запроса** — определяет, что делать с ресурсом.
- **Заголовки (Headers)** — дополнительные параметры запроса.
- **Тело запроса (Body)** — данные, которые передаются серверу (например, JSON).

HTTP запрос


Simple http request
URL of document on server
GET /reports/report-06-2016 HTTP/1.0
Host: ru.victor.org
Method
GET - requests resource
POST - Submits data to be processed
DELETE - deletes specific resource
...
Name of web-server
Version of http protocol

Метод	Описание
GET	Получение данных (без изменений).
HEAD	Запрос метаданных ресурса (без тела ответа).
POST	Создание нового ресурса.
PUT	Полное обновление ресурса.
PATCH	Частичное обновление ресурса.
DELETE	Удаление ресурса.
CONNECT	Создание туннеля к серверу.
OPTIONS	Получение информации о доступных методах.
TRACE	Отладочный запрос, возвращает исходный запрос.

Методы HTTP/HTTPS

Метод	Цель	Идемпотентность	Тело запроса	Примеры использования
GET	Получение ресурсов	✅ Да	❌ Не используется	Получение данных, напр. `GET /users`
POST	Создание ресурсов	❌ Нет	✅ Обязательно	Создание данных, напр. `POST /users`
PUT	Полное обновление ресурса	✅ Да	✅ Обязательно	Перезапись данных, напр. `PUT /users/1`
PATCH	Частичное обновление ресурса	✅ Да	✅ Обязательно	Обновление полей, напр. `PATCH /users/1`
DELETE	Удаление ресурсов	✅ Да	❌ Редко используется	Удаление данных, напр. `DELETE /users/1`

IT CAREER HUB

ВОПРОСЫ

IT CAREER HUB



Заголовки HTTP-запроса

Примеры заголовков

- **Accept** — какой тип данных ожидаем от сервера.
- **Content-Type** — формат данных, передаваемых в теле запроса.
- **User-Agent** — информация о клиенте (браузере, устройстве).

Из чего состоит HTTP-ответ?

- Заголовки (ключ: значение).
- Статус-код (трёхзначное число).
- Тело ответа (Body) — данные, полученные с сервера (может отсутствовать).

Статус-коды HTTP

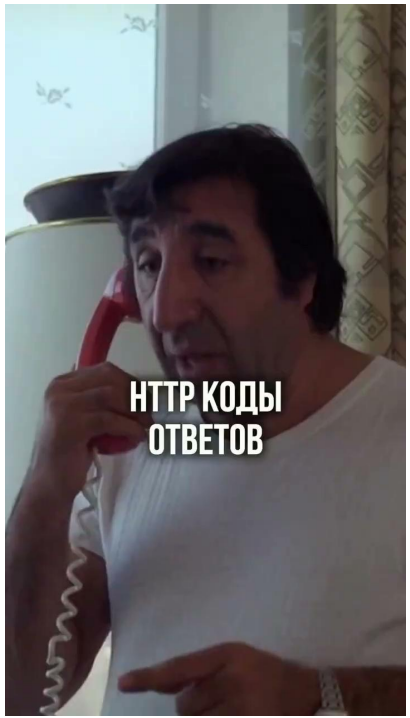
IT CAREER HUB

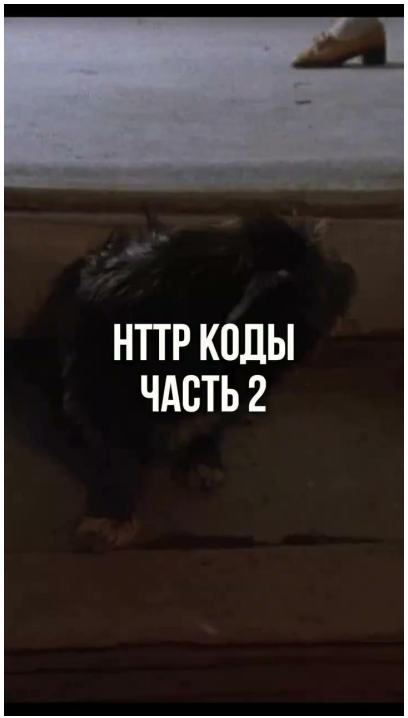
Категория	Описание	Примеры кодов
1xx	Информационные	100, 101, 199
2xx	Успешные запросы	200 (OK), 201 (Created)
3xx	Перенаправления	301 (Moved Permanently), 304 (Not Modified)
4xx	Ошибки клиента	400 (Bad Request), 404 (Not Found)
5xx	Ошибки сервера	500 (Internal Server Error), 503 (Service Unavailable)

Примеры

- 200 OK — успешный запрос.
- 404 Not Found — ресурс не найден.
- 500 Internal Server Error — ошибка на сервере.

HTTP СТАТУС КОДЫ





REST API



Набор рекомендаций по организации запросов и ответов.

Примеры REST-запросов

IT CAREER HUB

Метод	Путь	Описание
POST	/user/createWithArray	Создание списка пользователей.
GET	/user/{username}	Получение информации о пользователе.
PUT	/user/{username}	Обновление пользователя.
DELETE	/user/{username}	Удаление пользователя.
GET	/user/login	Вход в систему.
GET	/user/logout	Выход из системы.
POST	/user	Создание пользователя.

Описание сервиса



X-Clients — это сервис записи на приём к профильным специалистам. В нём есть четыре модуля, выполняющие разные операции.

Основные модули

- **auth** — авторизация пользователей
- **company** — работа с компаниями
- **employee** — работа с сотрудниками
- **superuser** — административные операции

Основные модули



auth

- Авторизация пользователей.

company

- Получение списка компаний.
- Добавление новой компании.
- Получение информации о компании.
- Изменение информации о компании.
- Изменение статуса компании.
- Удаление компании

Основные модули



employee

- Получение списка сотрудников компании.
- Добавление нового сотрудника.
- Получение информации о сотруднике по id.
- Изменение информации о сотруднике.

superuser

- Пересоздание списка компаний и пользователей.
- Очистка таблицы.
- Добавление 5 компаний и 15 пользователей.

Важно



Разработчики определяют, какие методы используются в запросах. Иногда это может противоречить стандартам. Например, во второй курсовой работе, чтобы удалить личное событие, нужно отправить **POST-запрос**, а не привычный **DELETE**.

Получение списка компаний



В новой версии X-Clients неавторизованные пользователи могут отправлять GET и POST-запросы.

Запрос на получение списка компаний



- **Метод:** GET
- **URL:** `http://5.101.50.27:8000/company/list`
- **Тело запроса:** пустое

```
[
  {
    "name": "QA Студия 'ТестировщикЪ'",
    "description": "Качественное тестирование для успешных проектов",
    "is_active": true
  },
```

Пример ответа (список компаний в JSON)

```
{
  "name": "Автоматизация тестирования",
  "description": "Эффективные решения для автоматизации процессов QA",
  "is_active": true
},
```

Пример ответа (список компаний в JSON)

```
{
  "name": "Консалтинговая компания 'QA-Эксперт'",
  "description": "Экспертиза и аудит качества программного обеспечения",
  "is_active": true
},
```

Пример ответа (список компаний в JSON)

```
  {
    "name": "Служба поддержки QA",
    "description": "Помощь и поддержка в вопросах тестирования",
    "is_active": false
  }
]
```

Пример ответа (список компаний в JSON)

Авторизация в системе



Для авторизации нужно отправить **POST-запрос** с логином и паролем.

Запрос на авторизацию



- **Метод:** POST
- **URL:** `http://5.101.50.27:8000/auth/login`
- **Тело запроса (JSON):**

```
{
  "username": "harrypotter",
  "password": "expelliarmus"
}
```

Запрос на авторизацию

```
{
  "user_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "role": "admin",
  "display_name": "harrypotter"
}
```

Пример ответа (токен авторизации)

Запрос на авторизацию



Токен (`user_token`) нужно передавать в каждом запросе, требующем авторизации.

Добавление новой компании



Чтобы создать компанию, отправляем **POST-запрос**.

Запрос на создание компании



- **Метод:** POST
- **URL:** `http://5.101.50.27:8000/company/create`
- **Тело запроса (JSON):**

```
{
  "name": "ITCH",
  "description": "курсы"
}
```

Добавление новой компании

```
{
  "id": 7,
  "name": "ITCH",
  "description": "курсы",
  "is_active": true
}
```

Пример ответа (созданная компания)

Запрос на создание компании



Теперь, если снова выполнить **GET-запрос** на список компаний, новая компания появится в списке.

Удаление компании



Чтобы удалить компанию, отправляем **DELETE-запрос**.

- **Метод:** DELETE
- **URL:** `http://5.101.50.27:8000/company/{{id}}?client_token={{ваш токен}}`
- **Тело запроса:** пустое

Удаление компании



Токен авторизации (`client_token`) указывает, какой пользователь выполняет удаление.

```
{
  "detail": "Компания успешно удалена",
  "company_id": 7
}
```

Пример ответа (подтверждение удаления)

Удаление компании



Если теперь снова отправить GET-запрос на список компаний, удалённая компания исчезнет.

Итог

- [x] Запросили список компаний и узнали их количество.
- [x] Авторизовались в системе.
- [x] Добавили новую компанию.
- [x] Запросили обновлённый список компаний и убедились, что новая компания появилась.
- [x] Удалили компанию.
- [x] Снова запросили список компаний и убедились, что новая компания исчезла.

IT CAREER HUB

ВОПРОСЫ

IT CAREER HUB



Библиотека Requests в Python

Requests



Это популярная библиотека в Python для отправки HTTP-запросов. Она упрощает работу с API, позволяя делать запросы (GET, POST, PUT, DELETE и другие) и обрабатывать ответы.

Установка библиотеки Requests



Команда

Чтобы установить requests, открой терминал или командную строку и введи.

Код

```
pip install requests
```

Установка библиотеки Requests



Команда

Проверить, установилась ли библиотека, можно командой.

Код

```
pip show requests
```

Установка библиотеки Requests



Команда

Если нужно установить конкретную версию.

Код

```
pip install requests==2.28.1
```

Установка библиотеки Requests



Команда

Для обновления библиотеки.

Код

```
pip install --upgrade requests
```

IT CAREER HUB

ВОПРОСЫ

ICH IT CAREER HUB



Получение списка компаний

Получение списка компаний



Отправим GET-запрос для получения списка компаний.

Запрос на получение списка компаний

- **Метод**: GET
- **URL**: `http://5.101.50.27:8000/company/list`

```python
import requests


def test_simple_req():
    resp = requests.get('http://5.101.50.27:8000/company/list')
    assert resp.status_code == 200  # Проверяем, что сервер вернул статус 200 (OK)
```

Пример кода для отправки запроса

Тест прошёл успешно, сервер вернул 200 OK



- Проверка типа контента в заголовке ответа
- Теперь убедимся, что сервер отправляет данные в формате JSON.

```python
import requests


def test_simple_req():
    resp = requests.get('http://5.101.50.27:8000/company/list')


    assert resp.status_code == 200  # Проверяем успешный статус
    assert resp.headers["Content-Type"] == "application/json"  # Проверяем тип контента
```

Пример кода для отправки запроса

Запрос успешен, сервер отправляет JSON



- **(Content-Type: application/json).**
- Проверка содержимого ответа
- Запишем тело ответа в переменную `response_body` и проверим, что название первой компании в списке соответствует `"QA Студия 'ТестировщикЪ'"`.

```python
import requests

def test_simple_req():

    resp = requests.get('http://5.101.50.27:8000/company/list')


    response_body = resp.json()  # Преобразуем ответ в JSON

    first_company = response_body[0]  # Получаем первую компанию из списка


    assert first_company["name"] == "QA Студия 'ТестировщикЪ'"

    assert resp.status_code == 200

    assert resp.headers["Content-Type"] == "application/json"
```

Проверка содержимого ответа

Все проверки пройдены



- Статус-код **200 (OK)**.
- Ответ в формате **JSON**.
- Первая компания имеет **правильное название**.

IT CAREER HUB

ВОПРОСЫ

IT CAREER HUB

Разница между text и json() в requests

Разница между text и json() в requests

Метод	Что возвращает?	Когда использовать?
`resp.text`	Строку (типа `str`)	Когда нужен текст в сыром виде
`resp.json()`	Python-объект (`dict` или `list`)	Когда сервер возвращает JSON

resp.text



- `resp.text` — получает текст в виде строки
- Этот метод возвращает тело ответа как строку.

```python
import requests

resp = requests.get("https://jsonplaceholder.typicode.com/posts/1")

print(resp.text)  # Получаем ответ в виде строки
```

Пример

```
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
  "body": "quia et suscipit..."
}
```


Вывод (str)

Важно



`resp.text` всегда возвращает строку, даже если это JSON.

resp.json()



- `resp.json()` — автоматически превращает JSON в Python-объект
- Если сервер возвращает `application/json`, `json()` преобразует ответ в словарь (`dict`) или список (`list`).

```python
import requests

resp = requests.get("https://jsonplaceholder.typicode.com/posts/1")

data = resp.json()  # Преобразуем JSON в Python-объект
print(data["title"])  # Теперь можно обращаться к данным как к словарю
```

Пример

```
sunt aut facere repellat provident occaecati excepturi optio reprehenderit
```

Вывод (dict)

```python
import requests


resp = requests.get("https://example.com")  # HTML, не JSON

print(resp.json())  # Ошибка: JSONDecodeError
```

Если сервер не отправляет JSON,
resp.json() вызовет ошибку

Когда использовать?



- Если работаешь с JSON API → `resp.json()` (удобно работать как со словарём).
- Если ответ сервера — обычный текст → `resp.text` (чтобы получить строку без обработки).

Итог



- `resp.text` — строка (`str`), даже если это JSON.
- `resp.json()` — автоматически превращает JSON в `dict` или `list`.
- `resp.json()` не сработает, если ответ — не JSON.

IT CAREER HUB

ВОПРОСЫ

IT CAREER HUB

Практическая работа

Условие задания



Получить список активных компаний и проверить их количество

Описание задания

1. Отправить GET-запрос на получение списка компаний.
2. Проверить, что запрос успешен (код 200).
3. Отфильтровать только активные компании (`is_active: true`).
4. Проверить, что в списке есть хотя бы 3 активные компании.

```python
import requests


def test_get_active_companies():
    url = "http://5.101.50.27:8000/company/list"

    # Отправляем GET-запрос
    response = requests.get(url)

    # Проверяем, что сервер вернул статус 200
    assert response.status_code == 200, f"Ошибка: статус {response.status_code}"
```

Пример решения

```python
# Преобразуем JSON-ответ в список словарей
companies = response.json()


# Фильтруем только активные компании
active_companies = [company for company in companies if company["is_active"]]


# Проверяем, что активных компаний не меньше 3
assert len(active_companies) >= 3, f"Ожидалось >=3 активных компаний, но найдено {len(active_companies)}"


print(f"Тест пройден! Найдено {len(active_companies)} активных компаний.")
```

Пример решения

IT CAREER HUB

ВОПРОСЫ

IT CAREER HUB

API Reference

API Reference



- Документация Requests: Requests API
- Использование `base_url`
- Для удобства работы с API сохраним базовый URL сервиса в переменную

```
import requests

base_url = "http://5.101.50.27:8000"
```

base_url

base_url



Теперь в запросах не нужно указывать полный URL, а можно просто добавлять к `base_url` нужные пути.

Авторизация



Перед выполнением защищённых запросов нужно авторизоваться и получить токен.

Описание запроса

- **Метод:** `POST`
- **URL:** `base_url + '/auth/login'`
- **Тело запроса (JSON):**

```
{
    "username": "harrypotter",
    "password": "expelliarmus"
}
```

Авторизация

```
def test_auth():
    creds = {
        'username': 'harrypotter',
        'password': 'expelliarmus'
    }
    resp = requests.post(base_url + '/auth/login', json=creds)

    assert resp.status_code == 200
```

Пример кода

Авторизация



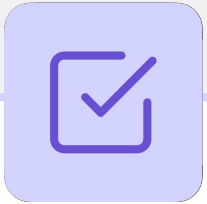
После запуска теста можно проверить объект `resp` в **Run and Debug**, чтобы увидеть токен авторизации.

```python
def test_auth():
    creds = {
        'username': 'harrypotter',
        'password': 'expelliarmus'
    }
    resp = requests.post(base_url + '/auth/login', json=creds)


    assert resp.json()["user_token"] is not None  # Проверяем, что токен получен
    assert resp.status_code == 200
```

Проверка наличия токена в ответе

Авторизация



После запуска теста можно проверить объект `resp` в **Run and Debug**, чтобы увидеть токен авторизации.

IT CAREER HUB

ВОПРОСЫ

IT CAREER HUB

Создание компании

Создание компании



Описание запроса

- **Метод:** POST
- **URL:** `base_url + '/company/create'`
- **Тело запроса (JSON):**

```
{
    "name": "python",
    "description": "requests"
}
```

Создание компании

```
def test_create_company():
    company = {
        "name": "python",
        "description": "requests"
    }
    resp = requests.post(base_url + '/company/create', json=company)

    assert resp.status_code == 201  # 201 означает, что компания успешно создана
```

Пример кода

Создание компании



Теперь можно легко создавать компании через API.

IT CAREER HUB

ВОПРОСЫ

IT CAREER HUB

Практическая работа

Условие задания



Написание негативного теста на отправку пустого тела при создании компании

Цель

Проверить, что при отправке пустого тела запроса на создание компании API:

- Возвращает статус-код **422 Unprocessable Entity**.
- В теле ответа содержит ошибку, где значение по ключу **"msg"** равно **"Field required"**.

```python
def test_create_company_empty_body():
    """Отправка пустого JSON"""
    resp = requests.post(base_url + '/company/create', json={})
    response = resp.json()
    assert resp.status_code == 422, f"Ожидался 422, получен {resp.status_code}"
    assert response["detail"][0]["msg"] == "Field required"
```

Пример решения

Условие задания



Проверка увеличения количества компаний после создания новой

Цель

Разработать автоматизированный тест, который:

1. Получает список всех компаний и запоминает его длину.
2. Создаёт новую компанию с помощью API.
3. Повторно получает список всех компаний.
4. Сравнивает количество компаний до и после создания, проверяя, что оно увеличилось на 1.

Шаги теста



1. Отправить **GET-запрос** на `base_url + '/company/list'`, чтобы получить список компаний.
2. Запомнить длину списка (**initial_count**).
3. Отправить **POST-запрос** на `base_url + '/company/create'` с новыми данными.
4. Повторно отправить **GET-запрос** на `base_url + '/company/list'`, чтобы получить обновлённый список.
5. Проверить, что длина списка увеличилась на 1.

```python
import requests


def test_create_company_increases_count():
    """Тест: создание компании увеличивает список компаний на 1"""
```

Пример решения

```python
# 1. Получаем текущий список компаний
resp = requests.get(base_url + "/company/list")
assert resp.status_code == 200, f"Ошибка: ожидался статус 200, получен {resp.status_code}"

companies_before = resp.json()  # Преобразуем ответ в JSON
initial_count = len(companies_before)  # Запоминаем количество компаний до создания новой
```

Пример решения

```
# 2. Создаём новую компанию
new_company = {
    "name": "Test Company",
    "description": "Automated test creation"
}
resp_create = requests.post(base_url + "/company/create", json=new_company)
assert resp_create.status_code == 201, f"Ошибка: ожидался статус 201, получен {resp_create.status_code}"
```

Пример решения

```python
# 3. Повторно получаем список компаний
resp_after = requests.get(base_url + "/company/list")
assert resp_after.status_code == 200, f"Ошибка: ожидался статус 200, получен {resp_after.status_code}"

companies_after = resp_after.json()  # Преобразуем ответ в JSON
final_count = len(companies_after)  # Запоминаем количество компаний после создания новой
```

Пример решения

```
# 4. Проверяем, что длина списка увеличилась на 1
assert final_count == initial_count + 1, f"Ожидалось {initial_count + 1} компаний, найдено {final_count}"

print(f"Тест пройден: до {initial_count}, после {final_count} (добавлена 1 компания).")
```

Пример решения

IT CAREER HUB

ВОПРОСЫ

IT CAREER HUB



Создание класса CompanyApi

Создание класса CompanyApi и использование его в тестах



- Создадим класс `CompanyApi`
- Создадим файл `company_api.py` и определим в нём класс `CompanyApi`, который будет отвечать за работу с API.

```python
import requests


class CompanyApi:
    """Класс для взаимодействия с API компаний"""

    def __init__(self, url):
        """Инициализация класса с базовым URL API"""
        self.url = url
```

Код company_api.py

```python
def get_company_list(self):
    """Получить список всех компаний"""
    resp = requests.get(self.url + '/company/list')
    assert resp.status_code == 200, f"Ошибка: ожидался статус 200, получен {resp.status_code}"
    return resp.json()
```

Код company_api.py

```python
def get_token(self, user, password):
    """Получить токен авторизации"""
    creds = {"username": user, "password": password}
    resp = requests.post(self.url + '/auth/login', json=creds)
    assert resp.status_code == 200, f"Ошибка: ожидался статус 200, получен {resp.status_code}"
    return resp.json()["user_token"]
```

Код company_api.py

```python
def create_company(self, name, description=""):
    """Создать новую компанию"""
    company = {"name": name, "description": description}
    resp = requests.post(self.url + '/company/create', json=company)
    assert resp.status_code == 201, f"Ошибка: ожидался статус 201, получен {resp.status_code}"
    return resp.json()
```

Код company_api.py

IT CAREER HUB

ВОПРОСЫ

ICH IT CAREER HUB

Зачем нужен CompanyApi?

Зачем нужен CompanyApi?



Этот класс **инкапсулирует** работу с API, что делает тесты **чище и удобнее**:

- Отдельные методы для получения списка компаний, авторизации и создания новых компаний.
- Упрощает написание и поддержку тестов.
- Можно переиспользовать в разных тестах, не копируя логику запросов.

Зачем нужен CompanyApi?



- Используем класс `CompanyApi` в тесте
- Теперь перепишем тест, используя `CompanyApi`.

```python
from company_api import CompanyApi  # Импортируем класс


# **Тест: проверка увеличения списка компаний после создания**
def test_create_company_increases_count():
    """Тест: создание компании увеличивает список компаний на 1"""

    base_url = "http://5.101.50.27:8000"
    api = CompanyApi(base_url)  # Инициализация API-клиента
```

Код теста

```
# 1. Получаем текущий список компаний
companies_before = api.get_company_list()
initial_count = len(companies_before)

# 2. Создаём новую компанию
api.create_company(name="Test Company", description="Automated test creation")
```

Код теста

```
    # 3. Повторно получаем список компаний
    companies_after = api.get_company_list()
    final_count = len(companies_after)

    # 4. Проверяем, что длина списка увеличилась на 1
    assert final_count == initial_count + 1, f"Ожидалось {initial_count + 1} компаний, найдено {final_count}"
    print(f"Тест пройден: до {initial_count}, после {final_count} (добавлена 1 компания).")

# Запуск теста
test_create_company_increases_count()
```

Код теста

Что изменилось?



Вынесли API в отдельный класс
Теперь взаимодействие с сервером сосредоточено в `CompanyApi`, а тест вызывает его методы.

Что изменилось?



Тест стал чище и удобнее

- Нет дублирования `requests.get()` и `requests.post()`.
- Все проверки (`assert resp.status_code`) теперь внутри `CompanyApi`.

Что изменилось?



Код легче поддерживать и расширять

- Если API изменится, **достаточно поменять только класс `CompanyApi`**, а не все тесты.
- Теперь тесты **проще в использовании и поддержке**.

Автоматический тест на получение компании по ID



Перед тем как написать тест, важно учесть несколько факторов:

- Использование фиксированного ID компании в тесте не рекомендуется, так как:
  - В списке может не оказаться ни одной компании (например, если кто-то очистил базу данных).
  - ID компаний могут изменяться, и заранее мы его не знаем.
- Надёжный тест должен сначала создать компанию, затем получить её ID и использовать его для проверки.

Добавим метод в CompanyApi



- Для получения информации о конкретной компании по её ID добавим новый метод в класс `CompanyApi`.
- Обновляем `company_api.py`

```python
import requests


class CompanyApi:
    """Класс для работы с API компаний"""

    def __init__(self, url):
        """Инициализация с базовым URL"""
        self.url = url
```

Обновляем company_api.py

```python
def get_company_list(self):
    """Получить список всех компаний"""
    resp = requests.get(self.url + '/company/list')
    assert resp.status_code == 200, f"Ошибка: ожидался статус 200, получен {resp.status_code}"
    return resp.json()
```

Обновляем company_api.py

```python
def create_company(self, name, description=""):
    """Создать новую компанию"""
    company = {"name": name, "description": description}
    resp = requests.post(self.url + '/company/create', json=company)
    assert resp.status_code == 201, f"Ошибка: ожидался статус 201, получен {resp.status_code}"
    return resp.json()
```

Обновляем company_api.py

```python
def get_company(self, company_id):
    """Получить информацию о компании по ID"""
    resp = requests.get(self.url + f'/company/{company_id}')
    assert resp.status_code == 200, f"Ошибка: ожидался статус 200, получен {resp.status_code}"
    return resp.json()
```

Обновляем company_api.py

Добавим метод в CompanyApi



Метод `get_company(self, company_id)` отправляет `GET`-запрос и возвращает информацию о компании. Если компания не найдена, API должно вернуть `404 Not Found`, но этот случай тестируем отдельно.

Напишем тест на создание и получение компании по ID



Создаём тест в `test_x_clients.py`

```python
def test_get_one_company():
    api = CompanyApi(base_url)
    # Создаем компанию
    name = "PyCharm"
    descr = "IDE"
    result = api.create_company(name, descr)
    new_id = result["id"]
```

Создаём тест в test_x_clients.py

```python
# Обращаемся к компании

new_company = api.get_company(new_id)


# Проверим название, описание и статус новой компании:

assert new_company["name"] == name

assert new_company["description"] == descr

assert new_company["is_active"] is True
```

Создаём тест в test_x_clients.py

Напишем тест на создание и получение компании по ID



Этот тест сначала создаёт новую компанию, затем получает её `ID` и использует его для проверки данных через API.

IT CAREER HUB

ВОПРОСЫ

ICH IT CAREER HUB

Изменения и удаления компании в API

Изменение компании



- Логика теста
- Чтобы изменить компанию, её сначала нужно создать. Мы создадим компанию, затем изменим её название и описание.

Добавим метод edit_company в CompanyApi



Этот метод:

- Получает токен авторизации.
- Формирует URL с ID компании и токеном.
- Отправляет `PATCH`-запрос с новыми данными.
- Возвращает обновлённую информацию о компании.

```python
import requests


class CompanyApi:
    """Класс для работы с API компаний"""

    def __init__(self, url):
        """Инициализация с базовым URL"""
        self.url = url
```

Обновляем company_api.py

```python
def get_token(self):
    """Получить токен авторизации (добавьте корректные данные)"""
    creds = {"username": "admin", "password": "admin"}
    resp = requests.post(self.url + '/auth/login', json=creds)
    assert resp.status_code == 200, f"Ошибка: ожидался статус 200, получен {resp.status_code}"
    return resp.json()["user_token"]
```

Обновляем company_api.py

```
def edit_company(self, company_id, new_name, new_descr):
    """Изменить название и описание компании по ID"""
    client_token = self.get_token()
    url_with_token = f"{self.url}/company/update/{company_id}?client_token={client_token}"


    company_data = {
        "name": new_name,
        "description": new_descr
    }
```

Обновляем company_api.py

```
resp = requests.patch(url_with_token, json=company_data)

assert resp.status_code == 200, f"Ошибка: ожидался статус 200, получен {resp.status_code}"

return resp.json()
```

Обновляем company_api.py

Удаление компании



Тест на удаление компании похож на тест изменения:

- Создаём компанию.
- Получаем её ID.
- Проверяем, что она существует.
- Удаляем компанию.
- Сравниваем количество компаний до и после удаления.
- Проверяем, что компания больше не доступна по ID.

Напишем тест test_delete_company



Этот тест:

- Создаёт компанию.
- Проверяет, что она существует.
- Удаляет компанию.
- Проверяет, что количество компаний уменьшилось.
- Проверяет, что компания больше не доступна.

```python
from company_api import CompanyApi


def test_delete_company():
    """Тест: удаление компании"""


    base_url = "http://5.101.50.27:8000"
    api = CompanyApi(base_url)
```

Добавляем тест в test_x_clients.py

```
# 1. Создаём компанию

name = "Company to be deleted"

descr = "Delete me"

result = api.create_company(name, descr)

company_id = result["id"]
```

Добавляем тест в test_x_clients.py

```
# 2. Получаем компанию по ID
new_company = api.get_company(company_id)


# 3. Проверяем, что компания создана корректно
assert new_company["name"] == name, f"Ожидалось '{name}', получено '{new_company['name']}'"
assert new_company["description"] == descr, f"Ожидалось '{descr}', получено '{new_company['description']}'"
assert new_company["is_active"] is True, "Ожидалось, что компания активна"
```

Добавляем тест в test_x_clients.py

```
# 4. Получаем список компаний и запоминаем количество
companies_before = api.get_company_list()
len_before = len(companies_before)

# 5. Удаляем компанию
api.delete_company(company_id)
```

Добавляем тест в test_x_clients.py

```python
# 6. Получаем список компаний после удаления
companies_after = api.get_company_list()
len_after = len(companies_after)

# 7. Проверяем, что список компаний уменьшился на 1
assert len_before - len_after == 1, f"Ожидалось {len_before - 1} компаний, найдено {len_after}"
```

Добавляем тест в test_x_clients.py

```
    # 8. Проверяем, что удалённая компания больше не доступна

    deleted = api.get_company(company_id)

    assert deleted["detail"] == "Компания не найдена", f"Ожидалось 'Компания не найдена', получено
'{deleted['detail']}'"


    print(f"Тест пройден: компания {company_id} успешно удалена.")
```

Добавляем тест в test_x_clients.py

IT CAREER HUB

ВОПРОСЫ

IT CAREER HUB

Тестирование деактивации и активации компании

Деактивация компании



Добавление метода `set_active_state` в `CompanyApi`

Этот метод:

- Получает токен авторизации.
- Формирует URL с ID компании и токеном.
- Отправляет `PATCH`-запрос, передавая `is_active` в теле запроса.
- Возвращает обновлённую информацию о компании.

```python
import requests


class CompanyApi:
    """Класс для работы с API компаний"""

    def __init__(self, url):
        """Инициализация с базовым URL"""
        self.url = url
```

Обновляем company_api.py

```python
def get_token(self):
    """Получить токен авторизации"""
    creds = {"username": "admin", "password": "admin"}
    resp = requests.post(self.url + '/auth/login', json=creds)
    assert resp.status_code == 200, f"Ошибка: ожидался статус 200, получен {resp.status_code}"
    return resp.json()["user_token"]
```

Обновляем company_api.py

```
def set_active_state(self, company_id, is_active):
    """Изменить статус активности компании"""
    client_token = self.get_token()
    url_with_token = f"{self.url}/company/status_update/{company_id}?client_token={client_token}"
```

Обновляем company_api.py

```
resp = requests.patch(url_with_token, json={"is_active": is_active})

assert resp.status_code == 200, f"Ошибка: ожидался статус 200, получен {resp.status_code}"

return resp.json()
```

Обновляем company_api.py

Тестирование деактивации компании



Этот тест:

- Создаёт новую компанию.
- Получает её ID.
- Вызывает `set_active_state()` с `False`.
- Проверяет, что компания стала неактивной.

```python
from company_api import CompanyApi


def test_deactivate_company():
    """Тест: деактивация компании"""

    base_url = "http://5.101.50.27:8000"
    api = CompanyApi(base_url)
```

Добавляем тест в test_x_clients.py

```
# 1. Создаём компанию

name = "Company to be deactivated"

result = api.create_company(name)

company_id = result["id"]


# 2. Деактивируем компанию

body = api.set_active_state(company_id, False)
```

Добавляем тест в test_x_clients.py

```
# 3. Проверяем, что статус изменился на "неактивный"
assert body["is_active"] is False, "Ожидалось, что компания будет неактивной"


print(f"Тест пройден: компания {company_id} успешно деактивирована.")
```

Добавляем тест в test_x_clients.py

IT CAREER HUB

ВОПРОСЫ

IT CAREER HUB

Практическая работа

Условие задания



Выполнить API-запрос с помощью библиотеки Requests

Используя библиотеку `requests`, выполнить `GET`-запрос к `https://reqres.in/api/users/2` и обработать ответ.

Требования

1. Отправить `GET`-запрос к API `https://reqres.in/api/users/2`.
2. Проверить, что запрос выполнен успешно (ожидаемый статус-код `200 OK`).
3. Извлечь `JSON`-данные из ответа и вывести следующую информацию:
    - **ID пользователя**
    - **Имя (`first_name`)**
    - **Фамилию (`last_name`)**
    - **Email**
    - **Аватар (`avatar`)** (ссылка на изображение.

```
{
    "data": {
        "id": 2,
        "email": "janet.weaver@reqres.in",
        "first_name": "Janet",
        "last_name": "Weaver",
        "avatar": "https://reqres.in/img/faces/2-image.jpg"
    },
```

Ожидаемый ответ API (пример JSON-ответа)

```
    "support": {
        "url": "https://reqres.in/#support-heading",
        "text": "To keep ReqRes free, contributions towards server costs are appreciated!"
    }
}
```

Ожидаемый ответ API (пример JSON-ответа)

```python
import requests

import pytest


BASE_URL = "https://reqres.in/api/users"


@pytest.mark.parametrize("user_id, expected_first_name, expected_last_name, expected_email", [
    (2, "Janet", "Weaver", "janet.weaver@reqres.in"),
])
```

Пример решения

```python
def test_get_existing_user(user_id, expected_first_name, expected_last_name, expected_email):
    """Тест: получение информации о существующем пользователе"""
    response = requests.get(f"{BASE_URL}/{user_id}")
```

Пример решения

```python
# Проверяем, что запрос выполнен успешно (код 200)
assert response.status_code == 200, f"Ожидался статус 200, получен {response.status_code}"


data = response.json()


# Проверяем, что в ответе есть ключ "data"
assert "data" in data, "Ответ API не содержит ключ 'data'"


user = data["data"]
```

Пример решения

```
    # Проверяем, что данные пользователя совпадают с ожидаемыми

    assert user["id"] == user_id, f"Ожидался ID {user_id}, получен {user['id']}"

    assert user["first_name"] == expected_first_name, f"Ожидалось '{expected_first_name}', получено
'{user['first_name']}'"

    assert user["last_name"] == expected_last_name, f"Ожидалось '{expected_last_name}', получено
'{user['last_name']}'"

    assert user["email"] == expected_email, f"Ожидалось '{expected_email}', получено '{user['email']}'"
```

Пример решения

```python
def test_get_non_existing_user():
    """Тест: получение информации о несуществующем пользователе"""
    user_id = 23  # Несуществующий пользователь
    response = requests.get(f"{BASE_URL}/{user_id}")
```

Пример решения

```python
# Проверяем, что API вернул 404 Not Found
assert response.status_code == 404, f"Ожидался статус 404, получен {response.status_code}"

# Проверяем, что тело ответа пустое
assert response.text == "{}", f"Ожидался пустой JSON, получен {response.text}"
```

Пример решения

IT CAREER HUB

ВОПРОСЫ

Домашнее задание

Разработать автоматические тесты, которые проверяют корректность работы API для управления сотрудниками.

Создайте класс `EmployeeApi` для создания вспомогательных методов.

API Методы

1. Создание нового работника

- **Метод:** `POST`
- **URL:** `http://5.101.50.27:8000/employee/create`
- **Описание:** Создаёт нового сотрудника, принимает данные в `JSON`.

Домашнее задание

2. Получение информации о работнике

- **Метод:** `GET`
- **URL:** `http://5.101.50.27:8000/employee/info`
- **Описание:** Получает данные о сотруднике по его `ID`.

3. Изменение данных о работнике

- **Метод:** `PATCH`
- **URL:** `http://5.101.50.27:8000/employee/change`
- **Описание:** Позволяет изменить информацию о сотруднике по его `ID`.

Заключение

